

In **Data Structures and Algorithms (DSA)**, a **tree** is a hierarchical data structure used to organize and store data in a parent-child relationship.

What is a Tree?

A **tree** is a collection of nodes where:

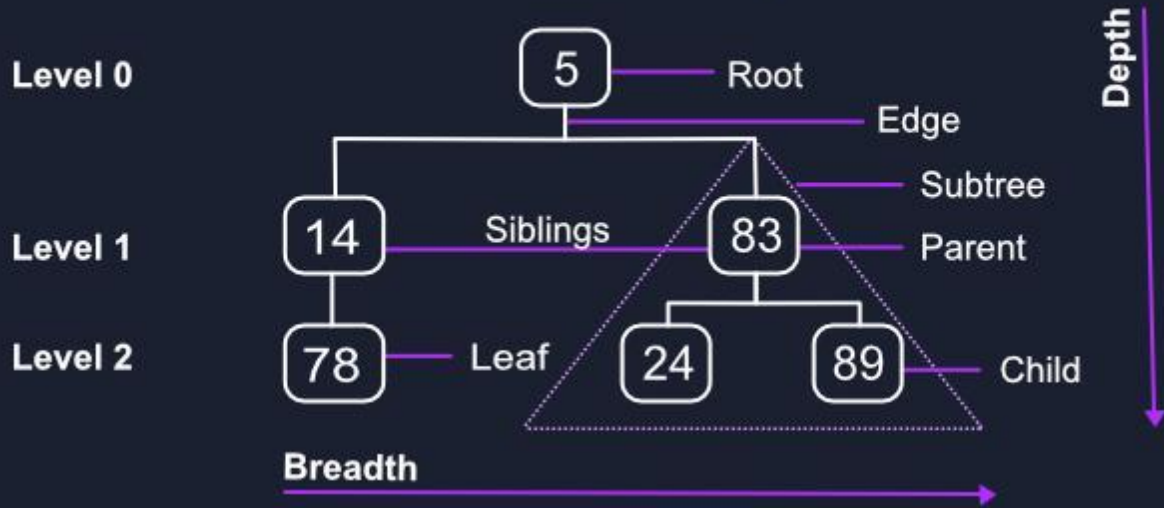
- One node is designated as the **root**
 - Every node can have **zero or more child nodes**
 - There are **no cycles** (unlike graphs)
 - Each node (except root) has exactly **one parent**
-

Basic Terminology

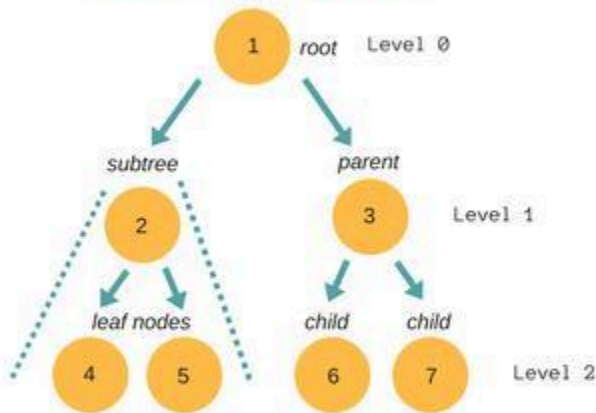
- **Root:** Topmost node of the tree
 - **Parent:** A node that has children
 - **Child:** A node connected below another node
 - **Leaf:** A node with no children
 - **Edge:** Connection between two nodes
 - **Height:** Longest path from root to a leaf
 - **Depth:** Distance from root to a node
-

Example Tree Structure

Tree Anatomy



Binary Tree



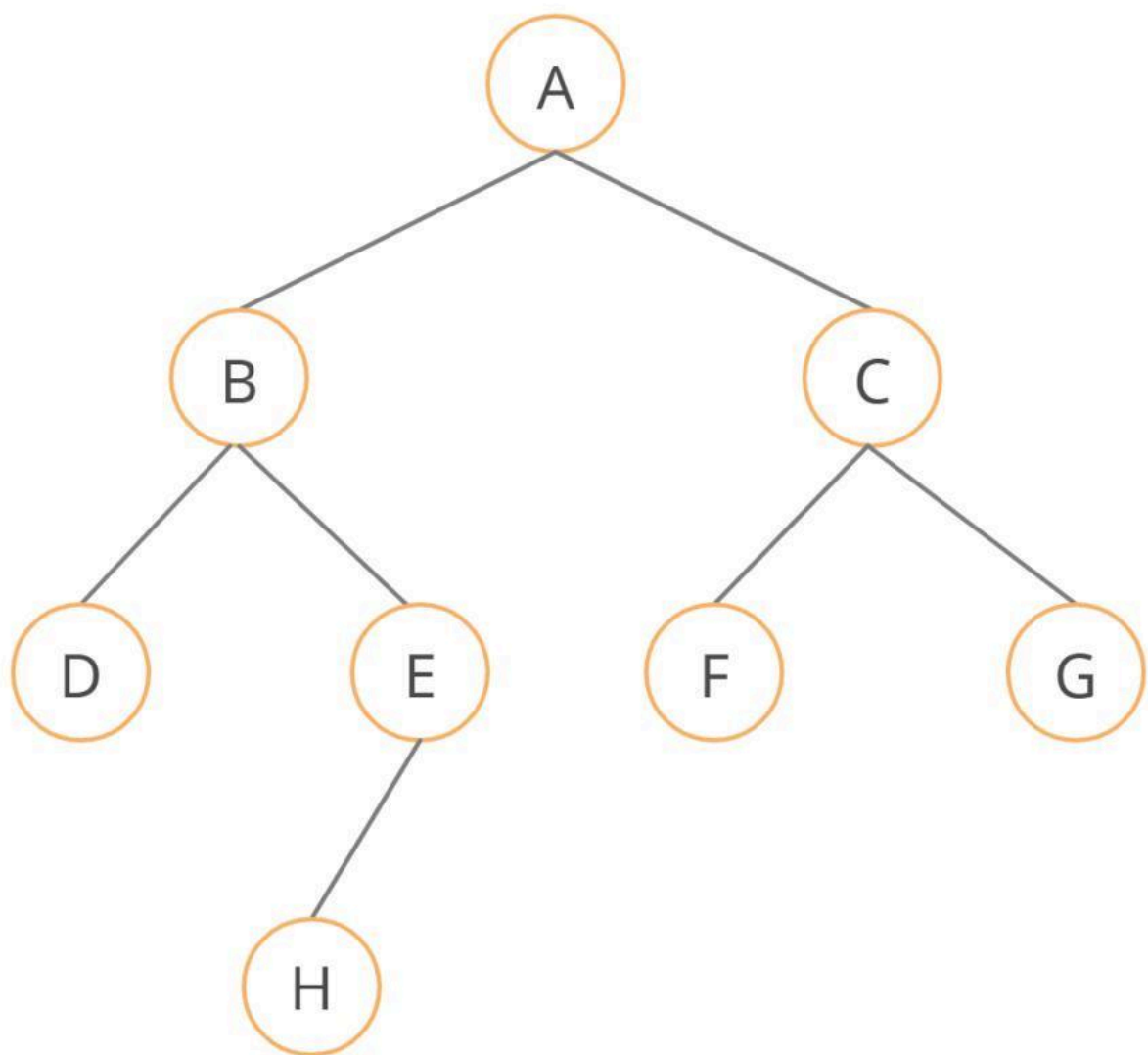
Legend: → edge ● node

The **root** node (1) is the first node of a tree and is also the parent of its left and right children (2, 3).

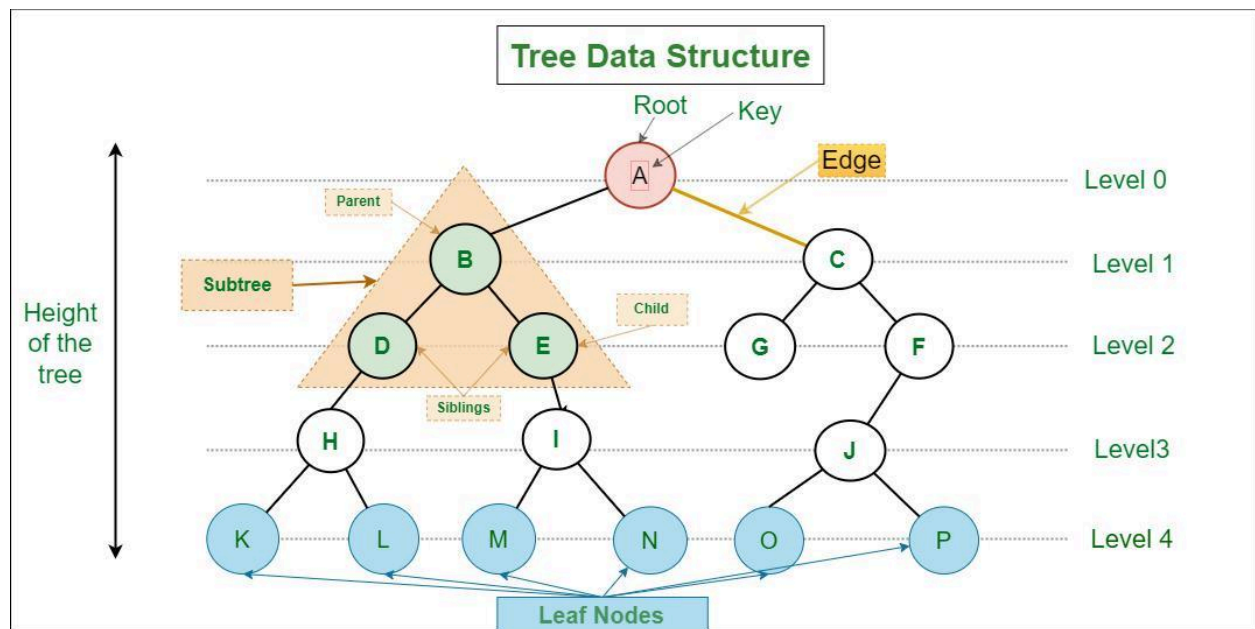
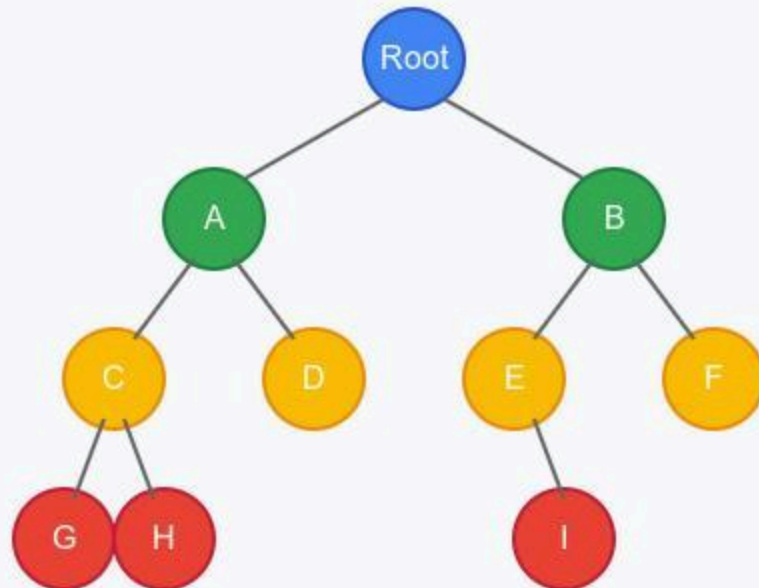
The links that connect a parent to its children are called **edges**.

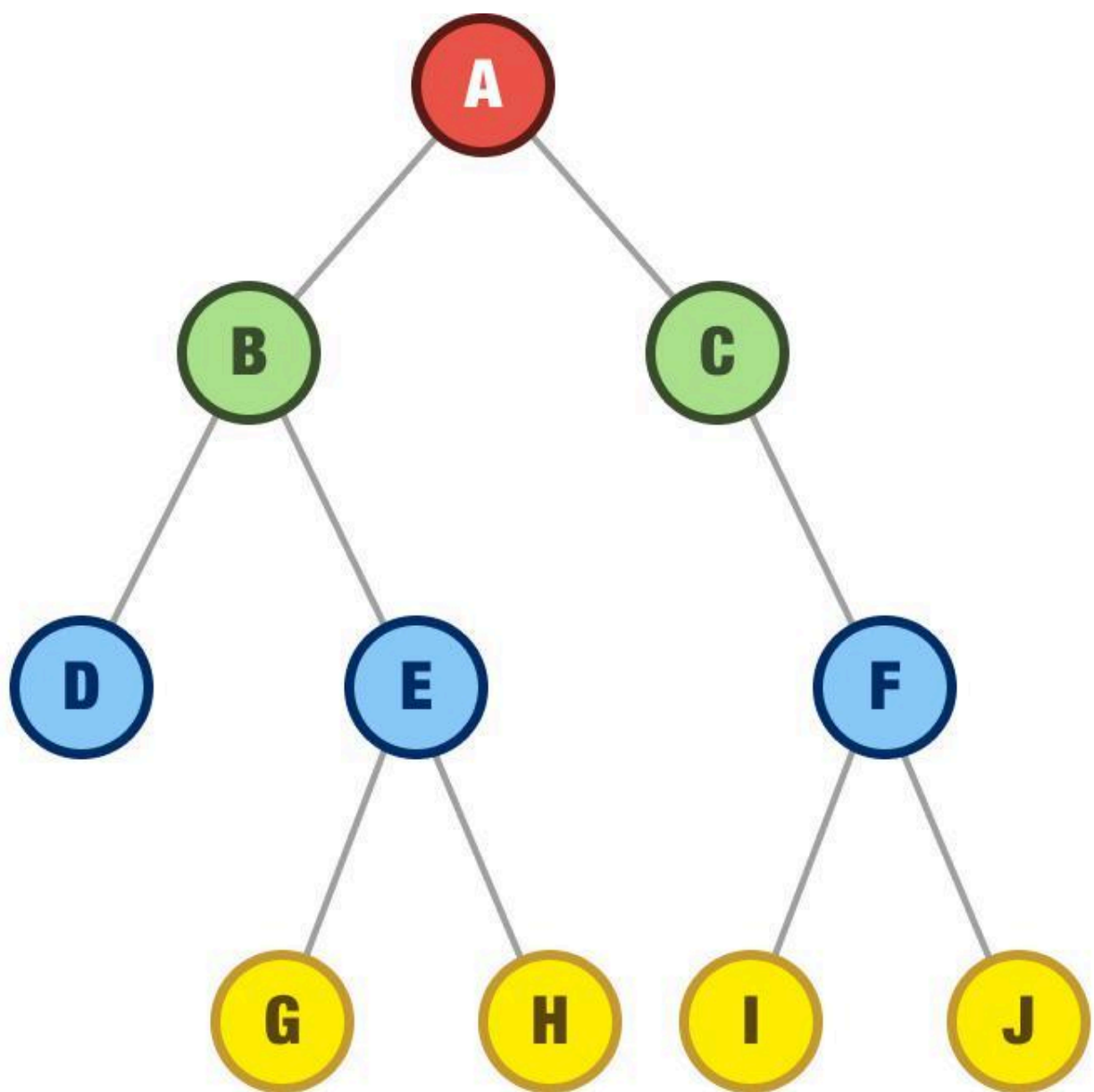
A tree can have multiple **subtrees**, which are composed of a parent's children.

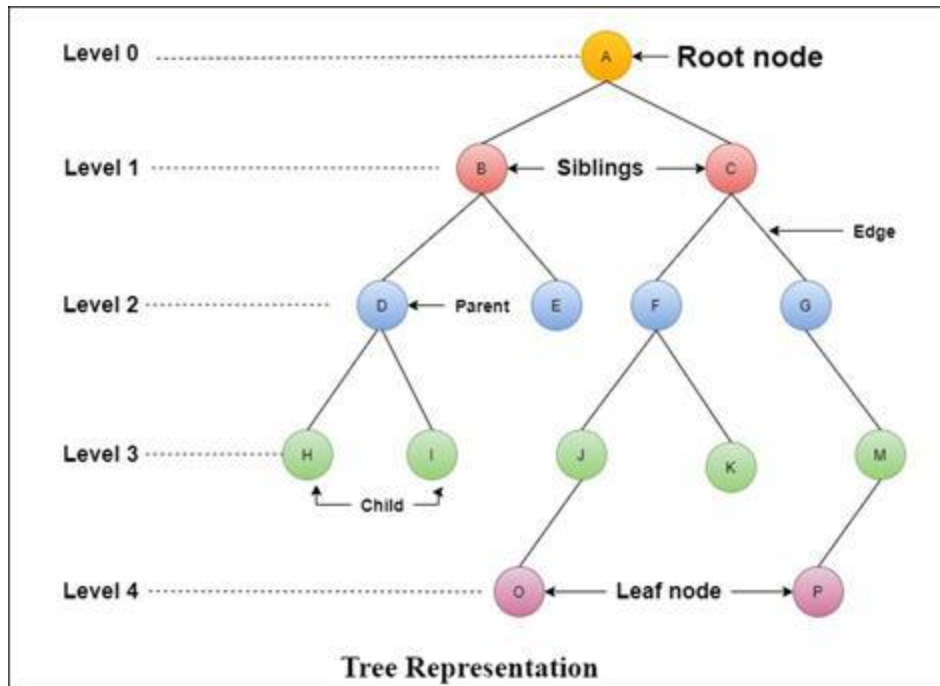
Trees have **leaf** nodes, which are nodes that have zero children.



Tree Data Structure







🌲 Types of Trees in DSA

1. **Binary Tree**
 - Each node has at most 2 children (left & right)
2. **Binary Search Tree (BST)**
 - Left child < Parent < Right child
3. **Balanced Trees**
 - Height is minimized (e.g., AVL Tree, Red-Black Tree)
4. **Heap**
 - Used in priority queues (Min-Heap / Max-Heap)
5. **Trie**
 - Used for strings (like dictionaries)
6. **N-ary Tree**
 - Nodes can have more than 2 children

⚙️ Common Tree Operations

- **Traversal**
 - Inorder (Left → Root → Right)
 - Preorder (Root → Left → Right)

- Postorder (Left → Right → Root)
 - **Insertion**
 - **Deletion**
 - **Searching**
-



Why Trees are Important

- Represent hierarchical data (file systems, org charts)
 - Efficient searching and sorting (BST)
 - Used in databases, networks, AI, compilers
-

If you want, I can explain **binary trees**, **BST with code**, or give **interview questions on trees**



This confusion is very common — the terms **height** and **depth** sound similar but mean different things. Let's make it crystal clear with a simple example.



Example Tree

```
A
 /\
B  C
 /\
D  E
```



Depth (of a node)

👉 **Depth = distance from ROOT to that node**

- Depth of **A** = 0 (root node)
- Depth of **B** = 1
- Depth of **C** = 1
- Depth of **D** = 2
- Depth of **E** = 2

👉 Formula idea:

Count how many edges from root → node

Height (of a node)

👉 Height = distance from that node to the deepest leaf

- Height of **D** = 0 (leaf node)
- Height of **E** = 0
- Height of **B** = 1
- Height of **C** = 0
- Height of **A** = 2

👉 Formula idea:

Count how many edges from node → deepest leaf

Key Difference (Super Easy Trick)

- Depth = going **DOWN** from root → node
 - Height = going **DOWN** from node → leaf
-

One-Line Memory Trick

- Depth → “How deep is this node from top?”
 - Height → “How tall is this node downwards?”
-

Important Note (Exam Trap)

Some books define:

- Height = number of **nodes** instead of edges

So:

- Always check if height is counted in **edges or nodes**

🔥 Final Summary

Concept	Direction	Starts From	Ends At
Depth	Top → Node	Root	Given node
Height	Node → Bottom	Given node	Deepest leaf

If you want, I can give you **tricky questions** or a **shortcut method to solve tree height/depth in exams quickly** 👍

Good — this is the **core part of trees** and very important for interviews. I'll explain each operation with one **clear example** so it sticks.

🌳 Example Tree (Binary Tree)

We'll use this tree:

```
  1
 / \
2   3
 / \
4   5
```

🔄 1. Tree Traversals

Traversal means **visiting all nodes in a specific order**.

✅ Inorder (Left → Root → Right)

👉 Rule:

Go LEFT → visit ROOT → go RIGHT

Steps:

- Go left of 1 → 2
- Go left of 2 → 4 → print 4
- Back to 2 → print 2
- Go right of 2 → 5 → print 5
- Back to 1 → print 1
- Go right → 3 → print 3

Output:

4 2 5 1 3

✅ **Preorder (Root → Left → Right)**

👉 Rule:

Visit ROOT first

Steps:

- Start at 1 → print 1
- Go left → 2 → print 2
- Go left → 4 → print 4
- Back → go right → 5 → print 5
- Back → go right → 3 → print 3

Output:

1 2 4 5 3

✅ **Postorder (Left → Right → Root)**

👉 Rule:

Visit ROOT at the end

Steps:

- Go left → 2
- Go left → 4 → print 4
- Go right → 5 → print 5
- Back → print 2
- Go right → 3 → print 3
- Finally → print 1

Output:

4 5 2 3 1

2. Insertion (Binary Search Tree example)

Let's insert values: 10, 5, 15, 3

Steps:

- 10 → root
- $5 < 10$ → go left
- $15 > 10$ → go right
- $3 < 10$ → left → 5 → $3 < 5$ → left

Tree:

```

  10
 /  \
5    15
/
3

```

3. Deletion (BST Example)

Delete 5:

Case:

Node has **one child (3)**

👉 Replace 5 with 3

New Tree:

```
  10
 /  \
3    15
```

4. Searching (BST)

Search **15**:

- Start at 10
 - $15 > 10 \rightarrow$ go right
 - Found 15 
-

Quick Summary

Operation	Idea
Inorder	Left $\rightarrow$ Root $\rightarrow$ Right
Preorder	Root $\rightarrow$ Left $\rightarrow$ Right
Postorder	Left $\rightarrow$ Right $\rightarrow$ Root
Insertion	Place node based on value
Deletion	Remove & adjust tree
Searching	Compare and move left/right

Pro Tip (Exam Trick)

- **Inorder of BST = sorted output**
- Preorder $\rightarrow$ used to **copy tree**

- Postorder → used in **deletion/free memory**
-

If you want, I can give:

- ✓ Code in C / Java / Python
- ✓ Trick to solve traversal in 10 seconds
- ✓ Most asked interview questions on trees

Let's break this down step-by-step so you **fully understand binary trees** 👉

What is a Binary Tree?

A **Binary Tree** is a tree data structure where:

👉 Each node can have **at most 2 children**

- Left child
 - Right child
-

Simple Example

```
  10
 /  \
5    20
/\   \
3 7   30
```

- 10 → root
 - 5 → left child of 10
 - 20 → right child of 10
 - 3, 7, 30 → leaf nodes
-

Types of Binary Trees (Important)

1. Full Binary Tree

👉 Every node has 0 or 2 children

```
  1
 / \
2   3
```

2. Complete Binary Tree

👉 All levels filled except possibly last, filled **left to right**

```
  1
 / \
2   3
/
4
```

3. Perfect Binary Tree

👉 All nodes have 2 children AND all leaves at same level

```
  1
 / \
2   3
/ \ / \
4 5 6 7
```

4. Skewed Tree

👉 Like a linked list (all nodes on one side)

```
  1
 \
  2
 \
  3
```



Where is Binary Tree Used?

This is **very important** for interviews 📌

✓ 1. Searching & Sorting

- Binary Search Tree (BST)

✓ 2. Heaps (Priority Queue)

- Used in scheduling, OS

✓ 3. Expression Trees

- Used in compilers
Example: $(a + b) * c$

✓ 4. Databases

- Indexing (B-Trees, similar concept)

✓ 5. AI / Decision Making

- Decision trees
-



Time Complexity of Binary Tree

It depends on the **height (h)** of the tree.



Searching

- Best case: $O(1)$ (root)
- Average case: $O(\log n)$ (balanced tree)
- Worst case: $O(n)$ (skewed tree)

Insertion

- Balanced tree $\rightarrow O(\log n)$
 - Skewed tree $\rightarrow O(n)$
-

Deletion

- Balanced $\rightarrow O(\log n)$
 - Worst $\rightarrow O(n)$
-

Traversals (Inorder, Preorder, Postorder)

👉 Always:

$O(n)$

Because every node is visited once

Important Insight (VERY IMPORTANT)

👉 Performance depends on **tree shape**

Tree Type	Time Complexity
-----------	-----------------

Balanced	$O(\log n)$
----------	-------------

Skewed	$O(n)$
--------	--------

Final One-Line Summary

- 👉 **Binary Tree = each node has max 2 children**
 - 👉 Used in **searching, heaps, compilers, AI**
 - 👉 Efficiency depends on **height of tree**
-

If you want next:

- ✅ Binary Search Tree (BST) in detail
- ✅ Code implementation (Java / C / Python)
- ✅ Interview questions on trees

Just tell me 👍

You've written the core rule correctly — now let's make it **clear, practical, and interview-ready**.

Balanced Trees

A **Balanced Tree** is a tree where the **height is minimized** so operations become faster.

Main idea:

Left subtree height $\approx$ Right subtree height

The difference between heights is small.

Why Balanced Tree is Needed?

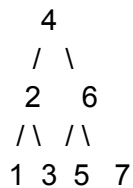
If tree becomes skewed:

```
1
 \
 2
  \
 3
  \
 4
```

Searching takes:
 $O(n)$

because it behaves like linked list.

Balanced Tree Example



Height is small.

Searching takes:
 $O(\log n)$

Balance Condition

For every node:

$$|h_{\text{left}} - h_{\text{right}}| \leq 1$$

Where:

- (h_{left}) = left subtree height
 - (h_{right}) = right subtree height
-

Properties of Balanced Tree

- ✓ Fast searching
- ✓ Fast insertion
- ✓ Fast deletion

- ✓ Small height
 - ✓ Efficient memory usage
-

Time Complexity

Operation	Complexity
-----------	------------

Search	$O(\log n)$
--------	-------------

Insert	$O(\log n)$
--------	-------------

Delete	$O(\log n)$
--------	-------------

Types of Balanced Trees

1. AVL Tree

Strictly balanced.

Condition:

Balance Factor = -1, 0, 1

Balance Factor:

$BF = h_{\text{left}} - h_{\text{right}}$

Uses rotations:

- LL Rotation
 - RR Rotation
 - LR Rotation
 - RL Rotation
-

2. Red-Black Tree

Less strict balancing.

Used in:

- Java TreeMap
- C++ STL map

Rules use:

- Red nodes
 - Black nodes
-

Real Life Uses

Application	Usage
Databases	Fast indexing
Search Engines	Fast lookup
File Systems	Directory storage
Compiler	Symbol tables

Interview MCQs

1. Main purpose of balanced tree?

- A. Increase height
 - B. Reduce height ☒
 - C. Remove nodes
 - D. Store duplicates
-

2. Search complexity in balanced BST?

- A. $O(n)$
- B. $O(\log n)$ ☒

- C. $O(n^2)$
 - D. $O(1)$
-

3. Which is balanced tree?

- A. AVL Tree ☒
 - B. Linked List
 - C. Stack
 - D. Queue
-

4. AVL Tree balance factor range?

- A. -5 to 5
 - B. 0 to 2
 - C. -1 to 1 ☒
 - D. 1 to 10
-

Easy Memory Trick

Balanced Tree =

Small Height = Fast Operations

Unbalanced Tree =

Large Height = Slow Operations



What is a Heap?

A **Heap** is a special type of **Binary Tree** that follows two rules:

1. Complete Binary Tree

- 👉 All levels are filled
 - 👉 Last level is filled **left to right**
-

2. Heap Property

There are two types:

Max Heap

- 👉 Parent $\geq$ Children

Example:

```
  50
 /  \
30   40
 /  \
10  20
```

- ✓ Largest element is always at the **root**
-

Min Heap

- 👉 Parent $\leq$ Children

Example:

```
  10
 /  \
20   30
 /  \
40   50
```

- ✓ Smallest element is always at the **root**

Why Heap is Used?

👉 To always get:

- **Maximum element quickly** (Max Heap)
 - **Minimum element quickly** (Min Heap)
-

Real Use Cases

- Priority Queue (VERY IMPORTANT 🔥)
 - CPU scheduling
 - Dijkstra's shortest path algorithm
 - Heap Sort
 - Task scheduling systems
-

Heap Operations

+ 1. Insertion

- 👉 Insert at last position
 - 👉 Then **heapify up** (swap with parent if needed)
-

✗ 2. Deletion (Remove Root)

- 👉 Replace root with last element
 - 👉 Then **heapify down**
-

3. Peek

- 👉 Get root (min/max)
 - 👉 Time: $O(1)$
-

Time Complexity

Operation	Time
Insert	$O(\log n)$
Delete	$O(\log n)$
Peek	$O(1)$
Heapify	$O(\log n)$

Important Insight

- 👉 Heap is **NOT** sorted completely

Example (Min Heap):

10 20 30 40 50

Only rule:

- Parent is smaller than children
 - But children are not necessarily sorted
-

Heap vs BST (Common Confusion)

Feature	Heap	BST
Order	Partial	Fully ordered
Search	$O(n)$ ❌	$O(\log n)$ ✅

Min/Max $O(1)$  $O(\log n)$

Memory Trick

👉 Heap = "Top element is always best (min/max)"

If you want next:

-  Heap code (Java / Python)
-  Heap sort step-by-step
-  Priority queue implementation

Just tell me 👍

 Tree Time Complexity Table				
Tree Type	Insert	Search	Delete	Update
Binary Tree (Normal)	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Binary Search Tree (BST)	$O(\log n)$ avg / $O(n)$ worst	$O(\log n)$ avg / $O(n)$ worst	$O(\log n)$ avg / $O(n)$ worst	$O(\log n)$ avg / $O(n)$ worst
Balanced Tree (AVL, Red-Black)	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$
Heap (Min / Max)	$O(\log n)$	$O(n)$	$O(\log n)$	$O(\log n)$

Got it — you don't just want definitions, you want to know **WHEN to use which tree in real problems**. That's exactly what interviewers expect 👍

1. Heap is which type of tree?

 Free offer 

- A. Full Binary Tree
- B. Complete Binary Tree 
- C. Skewed Tree
- D. BST

2. In Max Heap, root contains?

- A. Smallest element
- B. Largest element 
- C. Random element
- D. Middle element

3. Heap Sort uses?

- A. Queue
- B. Stack
- C. Heap 
- D. Graph

4. Time complexity of insertion in heap?

- A. $O(1)$
- B. $O(\log n)$ 
- C. $O(n)$
- D. $O(n^2)$



Trie Tree (Prefix Tree)

A Trie is a special tree used to store **strings/words** efficiently.

It is also called:

- Prefix Tree
- Digital Tree

Main Idea

Each character is stored in a node.

Example words:

cat
car
can



Trie structure:

```
root
|
c
|
a
/ | \
t r n
```



All words share common prefix:

Why Trie is Used?

Trie is used for:

- Fast word searching
- Autocomplete
- Dictionary
- Spell checker
- Search engine suggestions

Properties of Trie

- ✓ Stores characters
- ✓ Each path forms a word
- ✓ Common prefixes shared
- ✓ Efficient prefix searching

Example

1. Trie is mainly used for?

- A. Sorting
- B. Prefix searching ☒
- C. Arithmetic
- D. Graph traversal

2. Trie stores?

- A. Numbers only
- B. Characters ☒
- C. Floating values
- D. Arrays only

3. Time complexity of Trie search?

$O(m)$

Where:

- m = word length

4. Which application uses Trie?

- A. Autocomplete ☒
- B. Stack memory
- C. CPU scheduling

Problem Type	How HashMap is Used	Time Complexity
 Frequency Counting	Count occurrences using key → count	$O(n)$
 Find Duplicates	Store elements & check existence	$O(n)$
 Two Sum	Store value → index & check complement	$O(n)$
 Mapping / Pairing	Store key → value relation	$O(n)$ (build) / $O(1)$ lookup
 String Problems	Character → frequency mapping	$O(n)$
 Fast Searching	Direct lookup	$O(1)$ avg
 Memoization (DP)	Store computed results	$O(n)$

Spanning Tree

A **Spanning Tree** is a tree formed from a graph that:

- ✓ Connects all vertices
- ✓ Has no cycles
- ✓ Uses minimum edges required

Definition

For a graph with:

V vertices

A spanning tree contains:

$V - 1$

edges.

Example

Graph:

```
A
 /\
B---C
```

This graph has cycle.

Possible spanning tree:

```
A
 /\
B  C
```

OR

```
A---B
 \
  C
```

Both connect all vertices without cycle.

Properties of Spanning Tree

1. Connects all vertices

Every node must be reachable.

2. No cycles

If cycle exists → not a tree.

3. Minimum edges

For V vertices:

Edges = $V - 1$

4. Removing one edge disconnects graph

5. Adding one edge creates cycle

Types of Spanning Tree

1. Minimum Spanning Tree (MST)

A spanning tree with:

Minimum total edge weight

Used in weighted graphs.

Example

Graph:

A---B (2)

| |

(3) (1)

| |

C---D (4)

MST chooses edges with minimum cost.

Algorithms for MST

1. Prim's Algorithm

Builds MST step by step from a node.

Uses:

- Priority Queue
- Greedy approach

Complexity:

$O(E \log V)$

2. Kruskal's Algorithm

Chooses smallest edges first.

Uses:

- Sorting
- Union-Find (DSU)

Complexity:

$O(E \log E)$

Difference Between Tree and Spanning Tree

Feature	Tree	Spanning Tree
Structure	Independent	Derived from graph
Cycles	No	No
Vertices	May vary	Includes all graph vertices
Edges	$n-1$	$V-1$

Real Life Applications

Application	Usage
Network Design	Minimum cable cost
Roads/Railways	Minimum connection
Electrical Networks	Wiring optimization
Computer Networks	Routing
Telecommunication	Efficient connectivity

Important Interview MCQs

1. Spanning tree contains how many edges?

- A. V
 - B. $V+1$
 - C. $V-1$ ☒
 - D. $E-1$
-

2. Spanning tree contains?

- A. Cycles
 - B. Self loops
 - C. No cycles ✓
 - D. Directed edges only
-

3. Which algorithm is used for MST?

- A. Bubble Sort
 - B. Binary Search
 - C. Prim's Algorithm ✓
 - D. DFS only
-

4. MST stands for?

- A. Minimum Search Tree
 - B. Minimum Spanning Tree ✓
 - C. Maximum Search Tree
 - D. Minimum Sorting Tree
-

Easy Memory Trick

Spanning Tree =

Connect ALL nodes with NO cycle

Minimum Spanning Tree =

Minimum Cost Connection

Types of Spanning Tree

A **Spanning Tree** is a subgraph that:

- Connects all vertices

- Has no cycle
- Contains:

V-1

edges.

Main Types of Spanning Tree

There are mainly:

1. **Minimum Spanning Tree (MST)**
 2. **Maximum Spanning Tree**
 3. **Random Spanning Tree**
 4. **Directed Spanning Tree**
 5. **Shortest Path Tree**
-

1. Minimum Spanning Tree (MST)

A spanning tree with:

Minimum total edge weight

Example

A---B (2)

| |

3 1

| |

C---D (4)

Choose minimum cost edges:

AB = 2

BD = 1

AC = 3

Total:

6

Algorithms Used

Prim's Algorithm

Kruskal's Algorithm

Applications

- Network design
 - Cable wiring
 - Road construction
-

2. Maximum Spanning Tree

A spanning tree with:

Maximum total edge weight

Opposite of MST.

Usage

Used when:

- Strongest connection needed
- Maximum bandwidth network

3. Random Spanning Tree

Randomly selected spanning tree from graph.

Usage

- Network simulation
 - Probability models
 - Maze generation
-

4. Directed Spanning Tree

Used in directed graphs.

Also called:

Arborescence

All nodes reachable from root using directed edges.

Example

A → B
↓
C

Applications

- Routing
- Broadcasting

- Data flow systems

5. Shortest Path Tree (SPT)

Constructed using shortest paths from source node.

Difference from MST

MST	SPT
Minimum total cost	Minimum distance from source
Whole graph optimization	Single source optimization

Algorithms

Dijkstra Algorithm

Bellman-Ford

Comparison Table

Type	Goal
Minimum Spanning Tree	Minimum cost
Maximum Spanning Tree	Maximum cost
Random Spanning Tree	Random selection
Directed Spanning Tree	Directed connectivity
Shortest Path Tree	Shortest distance

Important Interview MCQs

1. MST stands for?

- A. Minimum Search Tree
 - B. Minimum Spanning Tree ☒
 - C. Maximum Spanning Tree
 - D. Minimum Sorted Tree
-

2. Which algorithm is used for MST?

- A. DFS
 - B. Bubble Sort
 - C. Kruskal's Algorithm ☒
 - D. Binary Search
-

3. Spanning tree contains cycle?

- A. Yes
 - B. No ☒
-

4. Number of edges in spanning tree?

V-1

Easy Memory Trick

MST

Minimum Cost Connection

SPT

Shortest Route from Source

Spanning Tree

Connect All Nodes Without Cycle

B Tree and B+ Tree (Simple Understanding)

Both **B Tree** and **B+ Tree** are advanced tree data structures mainly used in:

- Databases
- File systems
- Indexing

They are designed for:

Fast searching, insertion, deletion

especially for very large data stored on disk.

First Understand the Problem

Suppose database has:

10 crore records

Normal BST becomes slow because height increases.

So databases use:

- B Tree
- B+ Tree

because height remains very small.

1. B Tree

A **B Tree** is a self-balanced multi-way search tree.

Unlike BST:

1 node can store many keys

Example of B Tree

```
    [10 20]
   /  |  \
  [5] [15] [25 30]
```

Node contains:

- Multiple values
 - Multiple children
-

Properties of B Tree

- ✓ Balanced
 - ✓ Sorted data
 - ✓ Multiple children
 - ✓ Small height
 - ✓ Fast disk access
-

Search in B Tree

Search:

15

Start:

[10 20]

15 lies between:

10 and 20

Go middle child:

[15]

Found.

Complexity

Searching:

$O(\log n)$

Insertion:

$O(\log n)$

Deletion:

$O(\log n)$

2. B+ Tree

B+ Tree is improved version of B Tree.

Main difference:

All actual data stored only in leaf nodes

Internal nodes store only keys.

Example of B+ Tree

```
    [10 20]
   /  |  \
  [5] [15] [25]
```

Leaf Nodes:

[5] → [15] → [25]

Leaf nodes are connected like linked list.

Why B+ Tree is Better?

Because linked leaf nodes help:

- Fast sequential access
- Fast range queries

Example:

Find records between 1000 and 2000

B+ Tree is very efficient.

Difference Between B Tree and B+ Tree

Feature	B Tree	B+ Tree
Data Storage	Internal + Leaf nodes	Only leaf nodes

Leaf Nodes Linked	No	Yes 
Search Speed	Good	Faster
Range Queries	Slower	Faster
Used In	Some DB systems	Most databases

Real Life Usage

System	Tree Used
MySQL	B+ Tree
Oracle DB	B+ Tree
NTFS File System	B Tree/B+ Tree
Databases	B+ Tree

Easy Understanding

B Tree

Data everywhere

Internal nodes + leaf nodes contain data.

B+ Tree

Data only at bottom

Leaf nodes contain actual records.

Why Databases Prefer B+ Tree?

Because:

- Better caching
 - Faster range search
 - Sequential traversal easy
 - Less disk access
-

Interview MCQs

1. Which tree is mostly used in databases?

- A. BST
 - B. Heap
 - C. B+ Tree ☒
 - D. AVL
-

2. In B+ Tree, actual data is stored in?

- A. Root node
 - B. Internal node
 - C. Leaf node ☒
 - D. Parent node
-

3. B+ Tree leaf nodes are connected using?

- A. Stack
 - B. Queue
 - C. Linked List ☒
 - D. Array
-

4. Time complexity of searching in B Tree?

$O(\log n)$

Easy Memory Trick

B Tree

Data in ALL nodes

B+ Tree

Data only in LEAF nodes

In B tree they connected with parent and child pointer